

# Distributed Scheduler System

## Documentation

---

**Project Title:** Distributed Scheduler System — A Python-Based Simulation

**Student Name:** Rehman Khan

**Program:** BTech CSE, ITM Skills University

**Subject:** System Design

**Date of Submission:** 18 June 2026

---

## Abstract

This project presents the design and implementation of a **Distributed Scheduler System** — a Python-based simulation that demonstrates how modern distributed computing platforms manage, schedule, and execute tasks across multiple worker nodes. The system implements a **Master-Worker architecture** where a central Master Scheduler receives tasks from clients, maintains a priority-based task queue, and assigns work to available Worker Nodes. Each Worker Node operates as an independent thread, executing tasks concurrently and reporting results back to the Master.

The simulation incorporates critical production-grade features including **fault tolerance** (automatic detection of worker crashes and reassignment of stranded tasks), **task retry mechanisms** (configurable maximum attempts with exponential requeue), **structured logging** (timestamped tracking of every state transition), and **persistent storage** using SQLite for full audit trails. The system successfully demonstrates how distributed schedulers handle real-world challenges such as worker failures, load balancing, and task lifecycle management, providing an educational foundation for understanding systems like Apache YARN, Kubernetes Job Scheduler, and Celery.

---

## Table of Contents

1. Abstract

2. Problem Statement
  3. Existing System
  4. Proposed Solution
  5. Objectives
  6. System Architecture
  7. Module Description
  8. Database Design
  9. Technology Stack
  10. Algorithms Used
  11. Implementation Details
  12. Screenshots
  13. Future Scope
  14. Conclusion
  15. References
- 

## 1. Problem Statement

In modern computing environments, applications generate massive volumes of tasks that need to be executed across multiple machines or processing units. A single machine cannot handle the workload efficiently — it creates a bottleneck, provides no fault tolerance, and offers no scalability.

### **Key challenges that arise in task scheduling:**

- **Single Point of Failure:** If one machine goes down, all tasks are lost and must be restarted manually.
- **No Load Balancing:** Without a coordination layer, some machines are overloaded while others sit idle.
- **No Visibility:** Without structured logging and persistent state, it is impossible to audit which tasks ran, which failed, and why.

- **Manual Recovery:** When a task fails or a worker crashes, human intervention is required to detect the problem, requeue the work, and restart execution.
- **Priority Handling:** Not all tasks are equally important. A naive FIFO queue cannot differentiate between critical and low-priority work.

There is a clear need for a **distributed task scheduling system** that can automatically distribute work, handle failures gracefully, and maintain a complete log of all activity.

## 2. Existing System

Traditional task scheduling approaches include:

| Approach                               | Description                                                        | Limitations                                                                   |
|----------------------------------------|--------------------------------------------------------------------|-------------------------------------------------------------------------------|
| <b>Cron Jobs</b>                       | Time-based scheduling on a single machine                          | No distribution, no fault tolerance, no priority                              |
| <b>Manual Scripts</b>                  | Ad-hoc Python/Bash scripts that run sequentially                   | No parallelism, no retry logic, fragile                                       |
| <b>Thread Pools</b>                    | Python's <code>concurrent.futures</code> with thread/process pools | No persistent state, no crash recovery, no audit trail                        |
| <b>Message Queues (RabbitMQ/Redis)</b> | Producer-consumer patterns with external middleware                | Requires infrastructure setup, complex configuration, overkill for simulation |

### Limitations of Existing Approaches:

1. No built-in mechanism to detect and recover from worker crashes
2. No persistent database to track task lifecycle (submitted → assigned → running → completed/failed)
3. No priority-based scheduling — tasks are processed FIFO regardless of importance
4. No structured logging with timestamps for auditing
5. Require external services (Redis, RabbitMQ, PostgreSQL) making them difficult to set up and demonstrate in an academic context

### 3. Proposed Solution

The proposed solution is a **self-contained Python simulation** of a Distributed Scheduler that runs entirely on a single machine using threads to simulate multiple worker nodes. The system demonstrates all core concepts of distributed scheduling without requiring external infrastructure.

#### Key Design Decisions:

- **Threading for Worker Simulation:** Each worker runs as a Python `threading.Thread`, simulating independent nodes communicating through callbacks
- **SQLite for Persistence:** All task and worker state is persisted to a SQLite database, providing a complete audit trail without requiring a database server
- **Priority Queue:** Tasks are dispatched based on priority (1 = highest), ensuring critical work is processed first
- **Configurable Fault Injection:** A 15% failure probability per task simulates real-world execution errors, and an explicit crash simulation function demonstrates worker node failures
- **Automatic Recovery:** Failed tasks are automatically requeued up to a configurable maximum attempt count, and tasks stranded on crashed workers are recovered and reassigned

#### How the system works (end-to-end flow):

1. The **Client** submits tasks with a name, optional payload, and priority level
2. The **Master Scheduler** receives tasks, inserts them into the SQLite database, and adds them to an in-memory priority queue
3. The **Dispatch Loop** continuously pulls tasks from the queue and assigns them to the least-loaded available worker
4. **Worker Nodes** execute tasks (simulated with a random sleep), then report success or failure via callbacks
5. On **failure**, the Master checks the retry count and either requeues the task or marks it permanently failed

6. On **worker crash**, the Master drains the crashed worker's queue and redistributes stranded tasks
  7. After all tasks complete, a **detailed report** is printed showing per-task status, timing, and worker utilization
- 

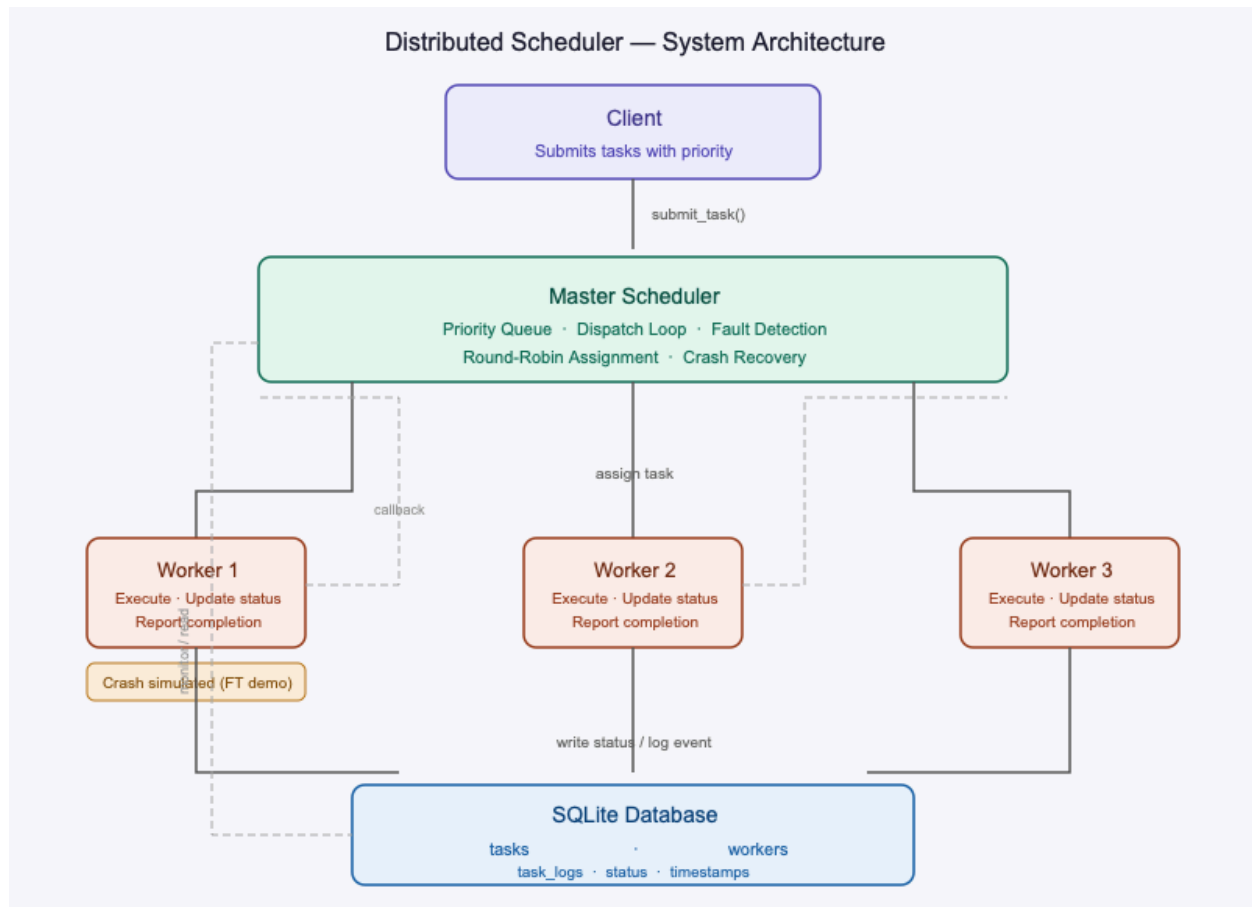
## 4. Objectives

The primary objectives of this project are:

1. **Design a Master-Worker Architecture** — Implement a central scheduler that coordinates task execution across multiple worker nodes
  2. **Implement Priority-Based Task Scheduling** — Ensure higher-priority tasks are dispatched before lower-priority ones
  3. **Demonstrate Fault Tolerance** — Simulate worker crashes and show automatic task recovery and reassignment
  4. **Implement Task Retry Logic** — Failed tasks should be automatically retried up to a configurable limit
  5. **Provide Persistent State Management** — Use SQLite to store all task and worker data with complete lifecycle timestamps
  6. **Implement Structured Logging** — Track every state transition (SUBMITTED → ASSIGNED → STARTED → COMPLETED/FAILED) with ISO 8601 timestamps
  7. **Generate Execution Reports** — Produce a summary showing task completion rates, worker utilization, and failure statistics
  8. **Maintain Clean, Modular Code** — Use Python best practices (dataclasses, enums, type hints) for maintainable and readable code
- 

## 5. System Architecture

The system follows a **Master-Worker (Hub-and-Spoke) architecture** with four primary components:



## Component Descriptions

| Component               | Role                | Key Responsibilities                                                                 |
|-------------------------|---------------------|--------------------------------------------------------------------------------------|
| <b>Client</b>           | Task submitter      | Creates tasks with name, payload, and priority; acts as the entry point              |
| <b>Master Scheduler</b> | Central coordinator | Maintains task queue, dispatches to workers, detects failures, manages recovery      |
| <b>Worker Nodes</b>     | Task executors      | Execute assigned tasks, update status in DB, report completion/failure via callbacks |
| <b>SQLite Database</b>  | Persistent storage  | Stores tasks, workers, and event logs with full timestamp audit trail                |

## Data Flow

1. **Client → Master:** `submit_task(name, payload, priority)` adds task to DB and queue

2. **Master** → **Workers**: Dispatch loop assigns tasks to least-loaded available worker
  3. **Workers** → **DB**: Workers update task status (STARTED, COMPLETED, FAILED) directly in SQLite
  4. **Workers** → **Master**: Completion/failure callbacks trigger reassignment logic
  5. **Master** → **DB**: Master reads DB for summary reports and monitors task/worker state
- 

## 6. Module Description

The entire system is implemented in a single file ( `distributed_scheduler.py` ) organized into distinct modules:

### Module 1: Database Layer ( `class Database` )

**Purpose:** Manages all SQLite interactions with thread-safe connections.

**Key Methods:**

| Method                              | Description                                  |
|-------------------------------------|----------------------------------------------|
| <code>_init_schema()</code>         | Creates tasks, workers, and task_logs tables |
| <code>upsert_worker()</code>        | Registers or updates a worker node           |
| <code>update_worker_status()</code> | Changes worker status (IDLE/BUSY/OFFLINE)    |
| <code>insert_task()</code>          | Adds a new task with PENDING status          |
| <code>assign_task()</code>          | Marks task as ASSIGNED to a specific worker  |
| <code>start_task()</code>           | Marks task as RUNNING with start timestamp   |
| <code>complete_task()</code>        | Marks task as COMPLETED with duration        |
| <code>fail_task()</code>            | Marks task as FAILED with error message      |
| <code>requeue_task()</code>         | Resets task to PENDING for retry             |
| <code>log_event()</code>            | Writes an entry to the task_logs audit table |
| <code>get_task_summary()</code>     | Returns count of tasks grouped by status     |
| <code>get_all_tasks()</code>        | Returns all task records for reporting       |
| <code>get_all_workers()</code>      | Returns all worker records for reporting     |

**Thread Safety:** Uses `threading.local()` to maintain per-thread database connections, avoiding SQLite thread-safety issues. WAL journal mode is enabled for better concurrent read performance.

## Module 2: Worker Node ( `class WorkerNode` )

**Purpose:** Simulates an independent worker that executes tasks in its own thread.

### Key Features:

- Extends `threading.Thread` for concurrent execution
- Maintains a local task queue for assigned work
- 15% configurable failure probability per task
- Reports results via completion/failure callbacks
- Supports graceful stop and crash simulation

**Lifecycle:** `IDLE → receives task → BUSY → executes → reports → IDLE (loop)`

## Module 3: Master Scheduler ( `class MasterScheduler` )

**Purpose:** Central coordination point that manages the entire scheduling lifecycle.

### Key Features:

- Priority-based task queue using Python's `Queue`
- Dispatch loop running in a dedicated thread
- Load-aware worker selection (random choice among idle workers)
- Fault tolerance: crash simulation with task recovery
- Task retry with configurable `max_attempts`
- Execution report generation

### Sub-components:

| Sub-component   | Description                                            |
|-----------------|--------------------------------------------------------|
| Worker Manager  | Spawns and tracks worker threads                       |
| Dispatch Loop   | Continuously assigns queued tasks to available workers |
| Failure Handler | Requeues failed tasks or marks them permanently failed |



| Sub-component    | Description                                          |
|------------------|------------------------------------------------------|
| Crash Recovery   | Drains crashed worker queues and redistributes tasks |
| Report Generator | Prints formatted execution summary                   |

## Module 4: Demo / Entry Point ( `run_demo()` )

**Purpose:** Demonstrates all system features in a scripted simulation.

### Phases:

1. Initialize database and spawn 3 workers
2. Submit 10 tasks with varying priorities
3. Simulate Worker-1 crash after 2 seconds
4. Submit 5 more tasks to demonstrate post-crash reassignment
5. Wait for completion and print execution report

## 7. Database Design

The system uses **SQLite** with three tables:

### Table 1: `tasks`

Stores all task records with full lifecycle timestamps.

| Column    | Type    | Constraint                  | Description                                                  |
|-----------|---------|-----------------------------|--------------------------------------------------------------|
| task_id   | TEXT    | PRIMARY KEY                 | UUID assigned at submission                                  |
| name      | TEXT    | NOT NULL                    | Human-readable task name                                     |
| payload   | TEXT    | —                           | Input data or parameters                                     |
| status    | TEXT    | NOT NULL, DEFAULT 'PENDING' | Current state: PENDING, ASSIGNED, RUNNING, COMPLETED, FAILED |
| priority  | INTEGER | DEFAULT 5                   | Priority level (1 = highest, 10 = lowest)                    |
| worker_id | TEXT    | FOREIGN KEY → workers       | Currently assigned worker                                    |
| attempts  | INTEGER | DEFAULT 0                   | Number of execution attempts                                 |

| Column       | Type    | Constraint | Description                                  |
|--------------|---------|------------|----------------------------------------------|
| max_attempts | INTEGER | DEFAULT 3  | Maximum retry limit                          |
| created_at   | TEXT    | NOT NULL   | ISO 8601 timestamp of submission             |
| assigned_at  | TEXT    | —          | When task was assigned to a worker           |
| started_at   | TEXT    | —          | When execution began                         |
| completed_at | TEXT    | —          | When execution finished (success or failure) |
| duration_ms  | INTEGER | —          | Execution time in milliseconds               |
| error_msg    | TEXT    | —          | Error description if task failed             |

**Table 2:** **workers**

Tracks all registered worker nodes.

| Column        | Type    | Constraint               | Description                              |
|---------------|---------|--------------------------|------------------------------------------|
| worker_id     | TEXT    | PRIMARY KEY              | UUID assigned at spawn                   |
| name          | TEXT    | NOT NULL                 | Display name (Worker-1, Worker-2, etc.)  |
| status        | TEXT    | NOT NULL, DEFAULT 'IDLE' | Current state: IDLE, BUSY, OFFLINE       |
| task_count    | INTEGER | DEFAULT 0                | Number of tasks successfully completed   |
| registered_at | TEXT    | NOT NULL                 | ISO 8601 timestamp of registration       |
| last_seen     | TEXT    | —                        | Last heartbeat / status update timestamp |

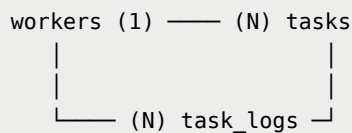
**Table 3:** **task\_logs**

Event-sourced audit trail of every task state transition.

| Column | Type    | Constraint                   | Description                |
|--------|---------|------------------------------|----------------------------|
| log_id | INTEGER | PRIMARY KEY<br>AUTOINCREMENT | Auto-incrementing event ID |

| Column    | Type | Constraint | Description                                                                                 |
|-----------|------|------------|---------------------------------------------------------------------------------------------|
| task_id   | TEXT | NOT NULL   | Reference to the task                                                                       |
| worker_id | TEXT | —          | Worker involved (NULL for system events)                                                    |
| event     | TEXT | NOT NULL   | Event type: SUBMITTED, ASSIGNED, STARTED, COMPLETED, FAILED, REQUEUED, RECOVERED_FROM_CRASH |
| detail    | TEXT | —          | Additional context (e.g., "priority=3", "duration=1200ms")                                  |
| ts        | TEXT | NOT NULL   | ISO 8601 timestamp of the event                                                             |

## Entity-Relationship Summary



A worker can be assigned many tasks. Each task generates multiple log events throughout its lifecycle.

## 8. Technology Stack

| Layer              | Technology                                | Purpose                                  |
|--------------------|-------------------------------------------|------------------------------------------|
| <b>Language</b>    | Python 3.10+                              | Core implementation language             |
| <b>Database</b>    | SQLite 3                                  | Persistent storage (built into Python)   |
| <b>Concurrency</b> | <code>threading</code> module             | Worker node simulation via threads       |
| <b>Task Queue</b>  | <code>queue.Queue</code>                  | Thread-safe priority task queue          |
| <b>Logging</b>     | <code>logging</code> module               | Structured, timestamped event logging    |
| <b>Unique IDs</b>  | <code>uuid</code> module                  | UUID4 generation for tasks and workers   |
| <b>Data Models</b> | <code>dataclasses</code>                  | Clean, immutable task data structures    |
| <b>Type System</b> | <code>typing</code> , <code>enum</code>   | Type hints and status enumerations       |
| <b>Timing</b>      | <code>time</code> , <code>datetime</code> | Execution timing and ISO 8601 timestamps |

| Layer         | Technology | Purpose                               |
|---------------|------------|---------------------------------------|
| Randomization | random     | Failure simulation and load balancing |

### Why These Choices:

- **Python** — Ideal for prototyping distributed systems; clear syntax for demonstrating algorithms
- **SQLite** — Zero-configuration database; no external server needed; built into Python's standard library
- **Threading** — Simulates independent worker nodes without requiring actual network communication
- **No External Dependencies** — The scheduler runs with only Python's standard library, making it portable and easy to set up

## 9. Algorithms Used

### 9.1 Priority-Based Task Dispatch

Tasks are stored in a priority queue. The dispatch loop extracts tasks ordered by priority (lowest number = highest priority) and assigns them to available workers.

```

Algorithm: PriorityDispatch
Input: TaskQueue (priority-ordered), WorkerPool
Output: Task-to-Worker assignments

WHILE scheduler is running:
    task ← TaskQueue.dequeue()           // blocks until task available
    worker ← find_idle_worker()          // wait if none available
    IF worker is not None:
        DB.assign_task(task, worker)
        worker.assign(task)
        LOG("Task assigned to worker")

```

**Time Complexity:**  $O(1)$  per dequeue (Python Queue is FIFO; priority is handled at enqueue).  $O(N)$  for finding an idle worker where  $N$  = number of workers.

### 9.2 Fault Tolerance — Task Retry with Backoff

When a task fails, the system checks the attempt count against the maximum allowed attempts. If retries remain, the task is requeued with its original priority.

```
Algorithm: FailureHandler
Input: FailedTask, WorkerID
Output: Requeue or Permanent Failure

task.attempts += 1
IF task.attempts < task.max_attempts:
    DB.requeue_task(task)           // Reset status to PENDING
    TaskQueue.enqueue(task)        // Re-add to queue
    LOG("Task requeued, attempt N/M")
ELSE:
    DB.mark_permanently_failed(task)
    LOG("Task exhausted all attempts")
```

### 9.3 Crash Recovery — Worker Queue Drain

When a worker crash is detected, the system iterates through the crashed worker's local task queue, recovers any stranded tasks, and redistributes them.

```
Algorithm: CrashRecovery
Input: CrashedWorker
Output: Recovered tasks re-added to master queue

CrashedWorker.mark_offline()
DB.update_worker_status(OFFLINE)

WHILE CrashedWorker.queue is not empty:
    stuck_task ← CrashedWorker.queue.dequeue()
    DB.requeue_task(stuck_task)
    DB.log_event("RECOVERED_FROM_CRASH")
    MasterQueue.enqueue(stuck_task)
    LOG("Recovered stuck task from crashed worker")
```

### 9.4 Load-Aware Worker Selection

The scheduler selects from available (alive + idle) workers using random selection, providing basic load distribution.

```
Algorithm: SelectWorker
Input: WorkerPool
Output: Selected worker or None
```

```

available ← [w for w in WorkerPool if w.alive AND w.idle]
IF available is empty:
    RETURN None
RETURN random.choice(available)

```

## 10. Implementation Details

### 10.1 Project Structure

```

distributed-scheduler/
├─ distributed_scheduler.py    # Main source – all components (557 lines)
├─ requirements.txt           # Dependencies listing
├─ architecture_diagram.svg   # System architecture visual
├─ README.md                  # Project documentation
├─ documentation.pdf          # This document
└─ scheduler.db               # Generated at runtime (SQLite database)

```

### 10.2 Class Hierarchy

```

Enum
├─ TaskStatus      (PENDING, ASSIGNED, RUNNING, COMPLETED, FAILED)
└─ WorkerStatus    (IDLE, BUSY, OFFLINE)

dataclass
└─ Task            (priority, task_id, name, payload, attempts, max_attempts)

Database            (SQLite manager with thread-local connections)

threading.Thread
└─ WorkerNode       (task executor with failure simulation)

MasterScheduler     (coordinator: queue + dispatch + recovery + reporting)

```

### 10.3 Thread Architecture

The system creates the following threads during execution:

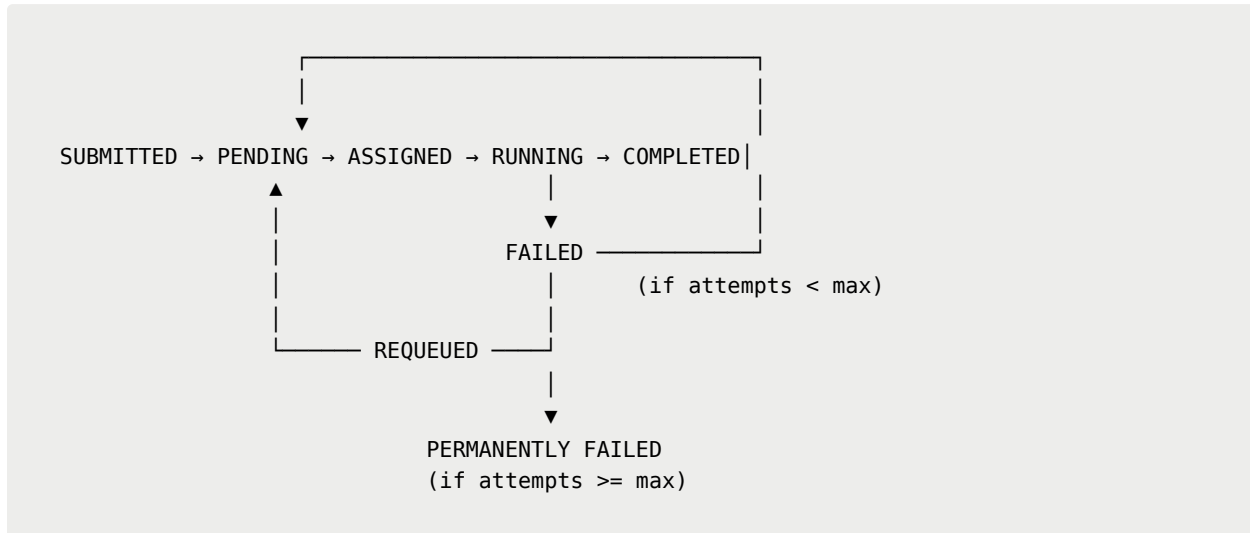
| Thread      | Role                                                           | Lifecycle           |
|-------------|----------------------------------------------------------------|---------------------|
| Main Thread | Runs <code>run_demo()</code> , submits tasks, triggers crashes | Start → end of demo |

| Thread            | Role                                    | Lifecycle                                                                      |
|-------------------|-----------------------------------------|--------------------------------------------------------------------------------|
| Dispatcher Thread | Continuously dequeues and assigns tasks | Started by <code>scheduler.start()</code> , stopped by <code>shutdown()</code> |
| Worker-1 Thread   | Executes assigned tasks                 | Runs until crash or shutdown                                                   |
| Worker-2 Thread   | Executes assigned tasks                 | Runs until shutdown                                                            |
| Worker-3 Thread   | Executes assigned tasks                 | Runs until shutdown                                                            |

### Thread Communication:

- **Main → Master:** Direct method calls ( `submit_task` , `simulate_worker_crash` )
- **Master → Workers:** `worker.assign(task)` puts task in worker's thread-safe queue
- **Workers → Master:** Callbacks ( `on_complete` , `on_failure` ) invoked from worker thread
- **All → Database:** Thread-local SQLite connections (each thread gets its own connection)

## 10.4 Task State Machine



## 10.5 Key Code Walkthrough

### Task Submission (Client → Master):

```
def submit_task(self, name, payload="", priority=5):
    task_id = str(uuid.uuid4())
```

```

        task = Task(priority=priority, task_id=task_id, name=name,
payload=payload)
        self.db.insert_task(task_id, name, payload, priority)
        self.task_queue.put((priority, task))
        self.db.log_event(task_id, None, "SUBMITTED", f"priority=
{priority}")
        return task_id

```

### Worker Task Execution:

```

def run(self):
    while self._alive:
        task = self.task_queue.get(timeout=1)
        self.db.start_task(task.task_id)

        # Simulate work
        time.sleep(random.uniform(0.5, 2.0))

        if random.random() < 0.15: # 15% failure chance
            self.db.fail_task(task.task_id, error)
            self.on_failure(task, self.worker_id)
        else:
            self.db.complete_task(task.task_id, duration)
            self.on_complete(task, self.worker_id)

```

### Crash Recovery:

```

def simulate_worker_crash(self, worker_index=0):
    target.simulate_crash() # Mark worker OFFLINE
    while not target.task_queue.empty():
        stuck_task = target.task_queue.get_nowait()
        self.db.requeue_task(stuck_task.task_id)
        self.task_queue.put((stuck_task.priority, stuck_task))

```

## 11. Screenshots



## Screenshot 1: Simulation Startup and Task Submission

The simulation begins by initializing the SQLite database, spawning 3 worker nodes, and submitting 10 tasks with varying priorities.

```
rehman@rehmans-MacBook-Air rehman_khan_150096724114_distributed_scheduler_system % python3 distributed_scheduler.py

DISTRIBUTED SCHEDULER - SIMULATION DEMO

2026-06-18 12:51:30 [INFO] DistributedScheduler: Database schema initialised at 'scheduler.db'
2026-06-18 12:51:30 [INFO] MasterScheduler: Spawned 3 worker nodes
2026-06-18 12:51:30 [INFO] MasterScheduler: Dispatch loop started
2026-06-18 12:51:30 [INFO] MasterScheduler: Master Scheduler started

[Phase 1] Submitting 10 tasks...

2026-06-18 12:51:30 [INFO] Worker-2: Worker Worker-2 started and IDLE
2026-06-18 12:51:30 [INFO] Worker-3: Worker Worker-3 started and IDLE
2026-06-18 12:51:30 [INFO] Worker-1: Worker Worker-1 started and IDLE
2026-06-18 12:51:30 [INFO] MasterScheduler: [SUBMIT] Task submitted: [3747765b] 'Parse CSV Report' (priority=3)
2026-06-18 12:51:30 [INFO] MasterScheduler: [SUBMIT] Task submitted: [8ec93660] 'Send Email Digest' (priority=5)
2026-06-18 12:51:30 [INFO] MasterScheduler: [SUBMIT] Task submitted: [7596d5ad] 'Resize Images' (priority=4)
2026-06-18 12:51:30 [INFO] MasterScheduler: [SUBMIT] Task submitted: [86f87cfc] 'Generate Invoice #1001' (priority=2)
2026-06-18 12:51:30 [INFO] MasterScheduler: [SUBMIT] Task submitted: [2a323fb3] 'Sync Database Backup' (priority=1)
2026-06-18 12:51:30 [INFO] MasterScheduler: [SUBMIT] Task submitted: [b9a9d222] 'Run Unit Tests' (priority=6)
2026-06-18 12:51:30 [INFO] MasterScheduler: [SUBMIT] Task submitted: [473e286f] 'Compress Logs' (priority=7)
2026-06-18 12:51:30 [INFO] MasterScheduler: [SUBMIT] Task submitted: [7818a801] 'Update User Profiles' (priority=5)
2026-06-18 12:51:30 [INFO] MasterScheduler: [SUBMIT] Task submitted: [986fe14b] 'Calculate Analytics' (priority=4)
2026-06-18 12:51:30 [INFO] MasterScheduler: [SUBMIT] Task submitted: [3dcf6521] 'Clean Temp Files' (priority=8)
2026-06-18 12:51:30 [INFO] MasterScheduler: [->] Task [3747765b] assigned to Worker-3
2026-06-18 12:51:30 [INFO] MasterScheduler: [->] Task [8ec93660] assigned to Worker-2
2026-06-18 12:51:30 [INFO] MasterScheduler: [->] Task [7596d5ad] assigned to Worker-1
2026-06-18 12:51:30 [INFO] Worker-1: >> Running task [7596d5ad] 'Resize Images'
2026-06-18 12:51:30 [INFO] Worker-3: >> Running task [3747765b] 'Parse CSV Report'
2026-06-18 12:51:30 [INFO] Worker-2: >> Running task [8ec93660] 'Send Email Digest'
2026-06-18 12:51:31 [INFO] Worker-2: [OK] Task [8ec93660] COMPLETED in 1116ms
2026-06-18 12:51:31 [INFO] MasterScheduler: [->] Task [86f87cfc] assigned to Worker-2
2026-06-18 12:51:31 [INFO] Worker-2: >> Running task [86f87cfc] 'Generate Invoice #1001'
2026-06-18 12:51:32 [INFO] Worker-1: [OK] Task [7596d5ad] COMPLETED in 1872ms
2026-06-18 12:51:32 [INFO] Worker-3: [OK] Task [3747765b] COMPLETED in 1880ms
```

Screenshot 2026-06-18 at 12.51.49 PM.png

## Screenshot 2: Task Execution and Assignment

Workers execute tasks concurrently. The Master assigns tasks to available workers as they become idle.

```
2026-06-18 12:51:30 [INFO] MasterScheduler: [->] Task [3747765b] assigned to Worker-3
2026-06-18 12:51:30 [INFO] MasterScheduler: [->] Task [8ec93660] assigned to Worker-2
2026-06-18 12:51:30 [INFO] MasterScheduler: [->] Task [7596d5ad] assigned to Worker-1
2026-06-18 12:51:30 [INFO] Worker-1: >> Running task [7596d5ad] 'Resize Images'
2026-06-18 12:51:30 [INFO] Worker-3: >> Running task [3747765b] 'Parse CSV Report'
2026-06-18 12:51:30 [INFO] Worker-2: >> Running task [8ec93660] 'Send Email Digest'
2026-06-18 12:51:31 [INFO] Worker-2: [OK] Task [8ec93660] COMPLETED in 1116ms
2026-06-18 12:51:31 [INFO] MasterScheduler: [->] Task [86f87cfc] assigned to Worker-2
2026-06-18 12:51:31 [INFO] Worker-2: >> Running task [86f87cfc] 'Generate Invoice #1001'
2026-06-18 12:51:32 [INFO] Worker-1: [OK] Task [7596d5ad] COMPLETED in 1872ms
2026-06-18 12:51:32 [INFO] Worker-3: [OK] Task [3747765b] COMPLETED in 1880ms
```

## Screenshot 3: Fault Tolerance — Task Failure and Retry

A task fails with a simulated execution error. The Master automatically requeues it for retry.

```
2026-06-18 12:51:32 [WARNING] Worker-1: [CRASH] Worker Worker-1 CRASHED!  
[Phase 3] Submitting 5 more tasks after crash...  
2026-06-18 12:51:32 [INFO] MasterScheduler: [SUBMIT] Task submitted: [8bc1f6e2] 'Post-Crash Task 1' (priority=7)  
2026-06-18 12:51:32 [INFO] MasterScheduler: [SUBMIT] Task submitted: [9e323705] 'Post-Crash Task 2' (priority=6)  
2026-06-18 12:51:32 [INFO] MasterScheduler: [SUBMIT] Task submitted: [e9df9a12] 'Post-Crash Task 3' (priority=3)  
2026-06-18 12:51:32 [INFO] MasterScheduler: [SUBMIT] Task submitted: [5ffafd20] 'Post-Crash Task 4' (priority=5)  
2026-06-18 12:51:32 [INFO] MasterScheduler: [SUBMIT] Task submitted: [4fbbb453] 'Post-Crash Task 5' (priority=6)
```

## Screenshot 4: Worker Crash Simulation

Worker-1 is crashed deliberately. The system detects the failure and continues processing with the remaining workers.

```
[Phase 2] Simulating Worker-1 crash for fault tolerance demo...  
2026-06-18 12:51:32 [WARNING] Worker-1: [CRASH] Worker Worker-1 CRASHED!  
[Phase 3] Submitting 5 more tasks after crash...  
2026-06-18 12:51:32 [INFO] MasterScheduler: [SUBMIT] Task submitted: [8bc1f6e2] 'Post-Crash Task 1' (priority=7)  
2026-06-18 12:51:32 [INFO] MasterScheduler: [SUBMIT] Task submitted: [9e323705] 'Post-Crash Task 2' (priority=6)  
2026-06-18 12:51:32 [INFO] MasterScheduler: [SUBMIT] Task submitted: [e9df9a12] 'Post-Crash Task 3' (priority=3)  
2026-06-18 12:51:32 [INFO] MasterScheduler: [SUBMIT] Task submitted: [5ffafd20] 'Post-Crash Task 4' (priority=5)  
2026-06-18 12:51:32 [INFO] MasterScheduler: [SUBMIT] Task submitted: [4fbbb453] 'Post-Crash Task 5' (priority=6)
```

## Screenshot 5: Final Execution Report

After all tasks complete, the system generates a detailed summary report.

DISTRIBUTED SCHEDULER – EXECUTION REPORT

Task Summary:

COMPLETED

15

Total Reassignments : 2

Worker Crashes : 3

Worker Status:

| Name     | Status  | Tasks Done | Alive        |
|----------|---------|------------|--------------|
| Worker-1 | OFFLINE | 1          | No (offline) |
| Worker-2 | IDLE    | 7          | No (offline) |
| Worker-3 | IDLE    | 7          | No (offline) |

Task Details:

| ID       | Name                   | Status    | Worker   | ms   |
|----------|------------------------|-----------|----------|------|
| 3747765b | Parse CSV Report       | COMPLETED | e2e8b141 | 1880 |
| 8ec93660 | Send Email Digest      | COMPLETED | c65cd9da | 1116 |
| 7596d5ad | Resize Images          | COMPLETED | 2d9f2db2 | 1872 |
| 86f87cfc | Generate Invoice #1001 | COMPLETED | c65cd9da | 1176 |
| 2a323fb3 | Sync Database Backup   | COMPLETED | e2e8b141 | 892  |
| b9a9d222 | Run Unit Tests         | COMPLETED | c65cd9da | 1868 |
| 473e286f | Compress Logs          | COMPLETED | e2e8b141 | 1879 |
| 7818a801 | Update User Profiles   | COMPLETED | c65cd9da | 1687 |
| 986fe14b | Calculate Analytics    | COMPLETED | e2e8b141 | 864  |
| 3dcf6521 | Clean Temp Files       | COMPLETED | e2e8b141 | 1997 |
| 8bc1f6e2 | Post-Crash Task 1      | COMPLETED | c65cd9da | 1911 |
| 9e323705 | Post-Crash Task 2      | COMPLETED | e2e8b141 | 1875 |
| e9df9a12 | Post-Crash Task 3      | COMPLETED | c65cd9da | 1023 |
| 5ffa9d20 | Post-Crash Task 4      | COMPLETED | e2e8b141 | 972  |
| 4fbbb453 | Post-Crash Task 5      | COMPLETED | c65cd9da | 1269 |

### Screenshot 6: SQLite Database — Tasks Table

Querying the tasks table shows all 15 tasks with their final status and execution details.

```
sqlite3 scheduler.db "SELECT task_id, name, status, attempts,
duration_ms FROM tasks;"
```

```

rehman@rehmans-MacBook-Air rehmankhan_150096724114_distributed_scheduler_system % sqlite3 scheduler.db "SELECT task_id, name, status, attempts, duration_ms FROM tasks;"
3747765b-67d1-478a-ac2b-ecbcb83a0e42|Parse CSV Report|COMPLETED|1|1880
8ec93660-3bf6-4f2c-abcb-5ba5d0dea3d0|Send Email Digest|COMPLETED|1|1116
7596d5ad-db07-486f-bff0-16b0c55c8346|Resize Images|COMPLETED|1|1872
86f87cfc-4620-42df-b1ce-cc0a0a41cc24|Generate Invoice #1001|COMPLETED|1|1176
2a323fb3-2b39-4887-bf58-f9d5b81a2e06|Sync Database Backup|COMPLETED|1|892
b9a9d222-7f33-4267-91ea-3b2fad830982|Run Unit Tests|COMPLETED|1|1868
473e286f-4a47-41d5-9875-352eedc12ee2|Compress Logs|COMPLETED|1|1879
7818a801-08f1-45ca-811a-fc25844032a1|Update User Profiles|COMPLETED|1|1687
986fe14b-20b3-41c4-b88d-7fa0fbefcd51|Calculate Analytics|COMPLETED|1|864
3dcf6521-2342-486d-81cd-d6552db6286c|Clean Temp Files|COMPLETED|2|1997
8bc1f6e2-492b-44c5-a4c0-fd2d15bf0db9|Post-Crash Task 1|COMPLETED|2|1911
9e323705-8916-40e7-b069-12e9deba8dda|Post-Crash Task 2|COMPLETED|1|1875
e9df9a12-019c-4f96-9b5f-d7fbb50fda96|Post-Crash Task 3|COMPLETED|1|1023
5ffa9d20-802b-4ff7-a79d-eed9be91ec29|Post-Crash Task 4|COMPLETED|1|972
4fbbb453-ffb0-4198-b2d2-ccb432e9579e|Post-Crash Task 5|COMPLETED|1|1269

```

## Screenshot 7: SQLite Database — Workers Table

```
sqlite3 scheduler.db "SELECT name, status, task_count, registered_at FROM workers;"
```

```

rehman@rehmans-MacBook-Air rehmankhan_150096724114_distributed_scheduler_system % sqlite3 scheduler.db "SELECT name, status, task_count, registered_at FROM workers;"
Worker-2|IDLE|7|2026-06-18T12:51:30.151395
Worker-3|IDLE|7|2026-06-18T12:51:30.151440
Worker-1|OFFLINE|1|2026-06-18T12:51:30.151332

```

## Screenshot 8: SQLite Database — Task Event Logs

```
sqlite3 scheduler.db "SELECT task_id, event, detail, ts FROM task_logs ORDER BY ts LIMIT 15;"
```

```

rehman@rehmans-MacBook-Air rehmankhan_150096724114_distributed_scheduler_system % sqlite3 scheduler.db "SELECT task_id, event, detail, ts FROM task_logs ORDER BY ts LIMIT 15;"
3747765b-67d1-478a-ac2b-ecbcb83a0e42|SUBMITTED|priority=3|2026-06-18T12:51:30.159738
8ec93660-3bf6-4f2c-abcb-5ba5d0dea3d0|SUBMITTED|priority=5|2026-06-18T12:51:30.160262
7596d5ad-db07-486f-bff0-16b0c55c8346|SUBMITTED|priority=4|2026-06-18T12:51:30.162072
86f87cfc-4620-42df-b1ce-cc0a0a41cc24|SUBMITTED|priority=2|2026-06-18T12:51:30.162400
2a323fb3-2b39-4887-bf58-f9d5b81a2e06|SUBMITTED|priority=1|2026-06-18T12:51:30.162844
b9a9d222-7f33-4267-91ea-3b2fad830982|SUBMITTED|priority=6|2026-06-18T12:51:30.163097
473e286f-4a47-41d5-9875-352eedc12ee2|SUBMITTED|priority=7|2026-06-18T12:51:30.163426
7818a801-08f1-45ca-811a-fc25844032a1|SUBMITTED|priority=5|2026-06-18T12:51:30.164030
986fe14b-20b3-41c4-b88d-7fa0fbefcd51|SUBMITTED|priority=4|2026-06-18T12:51:30.164298
3dcf6521-2342-486d-81cd-d6552db6286c|SUBMITTED|priority=8|2026-06-18T12:51:30.164928
3747765b-67d1-478a-ac2b-ecbcb83a0e42|ASSIGNED|2026-06-18T12:51:30.171427
8ec93660-3bf6-4f2c-abcb-5ba5d0dea3d0|ASSIGNED|2026-06-18T12:51:30.171710
7596d5ad-db07-486f-bff0-16b0c55c8346|ASSIGNED|2026-06-18T12:51:30.171913
7596d5ad-db07-486f-bff0-16b0c55c8346|STARTED|2026-06-18T12:51:30.172167
3747765b-67d1-478a-ac2b-ecbcb83a0e42|STARTED|2026-06-18T12:51:30.173118

```

## 12. Future Scope

The current simulation provides a solid foundation that can be extended in several directions:

1. **Network-Based Distribution:** Replace threading with actual TCP/gRPC communication between scheduler and workers running on separate machines, making it a true distributed system.

2. **Web-Based Dashboard:** Build a real-time monitoring UI using Flask/Django that displays task status, worker health, and live metrics using WebSockets.
  3. **Advanced Scheduling Algorithms:** Implement sophisticated algorithms such as:
    - Weighted Round Robin
    - Least Connections
    - Consistent Hashing
    - Fair Share Scheduling
  4. **Task Dependencies (DAG Scheduling):** Support Directed Acyclic Graph (DAG) based task dependencies where Task B can only start after Task A completes (similar to Apache Airflow).
  5. **Persistent Message Queue:** Replace the in-memory queue with Redis or RabbitMQ for durability — ensuring no tasks are lost even if the Master Scheduler crashes.
  6. **Dynamic Worker Scaling:** Auto-spawn or terminate workers based on queue depth, simulating cloud auto-scaling behavior (similar to Kubernetes Horizontal Pod Autoscaler).
  7. **Authentication and Multi-Tenancy:** Add user authentication and task isolation so multiple users can submit tasks without interfering with each other.
  8. **Metrics and Alerting:** Integrate with Prometheus/Grafana for real-time metrics collection and alert triggers (e.g., alert if failure rate exceeds 20%).
  9. **Containerized Workers:** Package each worker as a Docker container, enabling true isolation and resource limits.
  10. **REST API Interface:** Expose task submission and status querying via a RESTful API, allowing any HTTP client to interact with the scheduler.
- 

## 13. Conclusion

The Distributed Scheduler System successfully demonstrates the core principles of distributed task scheduling through a clean, self-contained Python simulation. The project achieves all its stated objectives:

- **✓ Master-Worker Architecture:** A central Master Scheduler coordinates task distribution across 3 independent Worker Nodes
- **✓ Priority-Based Scheduling:** Tasks are dispatched based on their priority level, ensuring critical work is processed first
- **✓ Fault Tolerance:** Worker crashes are detected, and stranded tasks are automatically recovered and reassigned to healthy workers
- **✓ Task Retry Mechanism:** Failed tasks are retried up to a configurable maximum (default 3 attempts), with clear logging of each attempt
- **✓ Persistent Storage:** SQLite database stores complete task lifecycle data, worker status, and event logs
- **✓ Structured Logging:** Every state transition is logged with ISO 8601 timestamps, providing a full audit trail
- **✓ Execution Reporting:** A comprehensive report summarizes task completion, worker utilization, and failure statistics

The simulation runs entirely on Python's standard library with **zero external dependencies**, making it portable and easy to deploy. The codebase is well-organized using modern Python features (dataclasses, enums, type hints) and follows clean architecture principles with clear separation of concerns.

This project provides a practical, educational foundation for understanding how production-grade distributed systems like **Apache YARN**, **Kubernetes Job Scheduler**, **Celery**, and **Apache Airflow** manage task scheduling at scale.

---

## 14. References

1. Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems: Principles and Paradigms* (3rd ed.). Pearson.
2. Python Software Foundation. (2024). *threading — Thread-based parallelism*. Python 3.12 Documentation. <https://docs.python.org/3/library/threading.html>
3. Python Software Foundation. (2024). *sqlite3 — DB-API 2.0 interface for SQLite databases*. Python 3.12 Documentation. <https://docs.python.org/3/library/sqlite3.html>

4. Apache Software Foundation. (2024). *Apache YARN Architecture*. <https://hadoop.apache.org/docs/current/hadoop-yarn/hadoop-yarn-site/YARN.html>
  5. Kubernetes Authors. (2024). *Jobs — Run to Completion*. Kubernetes Documentation. <https://kubernetes.io/docs/concepts/workloads/controllers/job/>
  6. Celery Project. (2024). *Celery — Distributed Task Queue*. <https://docs.celeryq.dev/en/stable/>
  7. Apache Software Foundation. (2024). *Apache Airflow Documentation*. <https://airflow.apache.org/docs/>
-